CS535 Project Proposal

GIA

Griffin Danehy, Ian McGregor

Based on the RISC-V ISA, the GIA (Griffin & Ian Architecture) is a general purpose instruction set designed to be a generalized yet effective architecture for teaching and observational purposes. Similar to RISC-V, GIA will also utilize a word size of 32 bits, a maximum operand per instruction of three, and a single instruction per word fetch paradigm. However, this ISA will also include its own unique features such as two sets of registers; one being a mixed register set for integer and special purpose registers and another smaller register set designed for floating point, On top of that, this design will also include an associative caching system that is able to be easily adaptable to fit the user's specific needs.

Specifically, the GIA will follow the von Neumann model for DRAM organization while also including a L1 cache to speed up any requests for loading or storing data. In terms of specifications, the cache will be unified, writethrough, no-allocate, with lines of four words each and feature N-set associativity with the value of N being able to be set by the user when first launching the simulator. Our simulator will also feature a true LRU replacement policy that will be implemented in C++ in which each line of the L1 cache is given an LRU field signaling its access time and an algorithm is tasked with updating the values of every line and evicting the last accessed one whenever space is needed for newer data.

In the same vein as other RISC architectures, GIA will feature all of the major categories of instruction types (ALU, control, shift, jump, compare, etc.) with the ALU instruction category featuring separate versions of several instructions depending on if the operands involved contain integers or floating points. For subroutine calls and leaf functions, a dedicated link register (ra) is created purely to store the return address and avoid having to access memory when said function has concluded. And for scenarios involving data that needs to be saved throughout multiple nested function calls, a dedicated register tracking the top of the stack will also be in place, allowing the simulator to be able to handle values being pushed to the stack and later popped out and returned once they're needed again. Finally, branches will be handled via a set of Branch-type instructions, which compare two registers and update the program counter if the result is true. We also include two condition code registers to handle special conditions present in either integer (overflow, divide by zero) or floating point values (overflows, underflows, NaN, infinity, etc.), leading to reduced pipeline stalling.

Instruction Set Breakdown:

Our instructions are broken down into 3-bit types, with each type defining the structure of the ensuing instruction. Within a type, each category has a 4-bit opcode. A category would be something like integer arithmetic, floating point arithmetic, or bitwise operations.

Then, within a category, each specific instruction uses funct3 and funct7 to differentiate itself. Within integer arithmetic, for example, integer addition and integer multiplication have unique funct3 values. Funct7 values, unless specified, are unused and therefore ignored and right now have no use. They are in place as extra bits for possible ISA extensions, as we progress through the project.

Below, each operation type will be broken down, and its categories listed, with each one's opcode in parentheses. Note that not all instructions within a type are required to use all of the components. For example, some types may have a second read-register, r2, that is not used in every instruction within that type. For those instructions, the rs2 bits are simply treated as unused. All immediate values are treated as signed integers.

Additionally, all registers use 5-bit addressing, despite floating-point registers not needing 5 bits. This is done for consistency. If an instruction's opcode signifies that floating-point registers are to be used, and the address is out of range, the instruction is invalid and treated as a NOP.

Register-Register operations (Type 000):

This type requires 2 read-registers and a destination register. It consists of largely arithmetic and bitwise operations. It is broken down as follows:

Type	Opcode	funct3	funct7	rs1	rs2	rd
3-bit	4-bit	3-bit	7-bit	5-bit	5-bit	5-bit

The following instructions are Register-Register type operations:

Code	**Common Name**	**Result**	**Funct3**
Arithmetic Instructions - Integer (Opcode 0):			
ADD	Integer Addition	rd = rs1 + rs2	000
SUB	Integer Subtraction	rd = rs1 - rs2	001
NEG	Integer Negation	rd = -rd	010

MUL	Integer Multiplication	rd = rs1 * rs2 (lower 32)	011
DIV	Integer Division	rd = rs1 / rs2 (floor division)	100
REM	Integer Remainder	rd = rs1 % rs2	101
All of these operations update the destination integer register, rd, and potentially the integer condition code. This would happen for overflows, underflows, or divide by zero errors. For example, if the multiplication result was greater than 2^32, MUL stores the lower 32 bits of the result in rd, and updates the condition code accordingly.			
Arithmetic Instructions - Floating Point (Opcode 1):			
FADD	Floating Point Addition	rd = rs1 + rs2	000
FSUB	Floating Point Subtraction	rd = rs1 - rs2	001
FNEG	Floating Point Negation	rd = -rd	010
FMUL	Floating Point Multiplication	rd = rs1 * rs2	011
FDIV	Floating Point Division	rd = rs1 / rs2	100
All of these operations update the destination floating point register, rd, and potentially the floating point condition code. This would happen for overflows, underflows, or divide by zero errors. Overflows and underflows would be stored in rd as positive or negative infinity, respectively.			
Bitwise Instructions (Opcode 2):			
AND	Bitwise AND	rd = rs1 & rs2	000
OR	Bitwise OR	rd = rs1 | r2	001
NOT	Bitwise NOT	rd = !rd	010
XOR	Bitwise XOR	rd = rs1 ^ rs2	011
All of these operations update the destination integer register, rd			
Compare and Store (Opcode 3):			
SEQ	Set Equal	rd = rs1 == rs2	000
SNEQ	Set Not Equal	rd = rs1 != rs2	001
SLT	Set Less Than	rd = rs1 < rs2	010
SGT	Set Greater Than	rd = rs1 > rs2	011

SLE	Set Less Or Equal	rd = rs1 <= rs2	100
SGE	Set Greater Or Equal	rd = rs1 >= rs2	101
All of these operations update the destination integer register, rd. rd is set to 1 if the comparison evaluates to true, and 0 otherwise.			
Copy Register (Opcode 4):			
CPY	Copy Register	rd = rs1	000
This operation copies the read-register rs1 into the destination register			

Immediate operations (Type 001):

This type requires an immediate value, one read-register, and a destination register. It is largely the supported immediate variations of type 000.

Type	Opcode	funct3	Immediate	rs1	rd
3-bit	4-bit	3-bit	12-bit	5-bit	5-bit

The following instructions are Immediate type operations:

Code	**Common Name**	**Result**	**Funct3**
Arithmetic Instructions - Integer Immediate (Opcode 0):			
ADDI	Immediate Integer Addition	rd = rs1 + imm	000
SUBI	Immediate Integer Subtraction	rd = rs1 - imm	001
MULI	Immediate Integer Multiplication	rd = rs1 * imm (lower 32)	010
DIVI	Immediate Integer Division	rd = rs1 / imm (floor division)	011
REMI	Immediate Integer Remainder	rd = rs1 % imm	100
All of these operations update the destination integer register, rd, and potentially the integer condition code. This would happen for overflows, underflows, or divide by zero errors. For example, if the multiplication result was greater than 2^32, MULI stores the lower 32 bits of the result in rd, and updates the condition code accordingly.			
Shift Immediate Instructions (Opcode 1):			

SLLI	Shift Left Logical Immediate	rd = rs1 << imm	000
SRLI	Shift Right Logical Immediate	rd = rs1 >>> imm	001
SRAI	Shift Right Arithmetic Immediate	rd = rs1 >> imm	010
These operations update the destination integer register, rd. Both logical operations fill the vacated bits with 0, while arithmetic right shift fills with the most significant bit.			
Bitwise Immediate Instructions (Opcode 2):			
ANDI	Immediate Bitwise AND	rd = rs1 & imm	000
ORI	Immediate Bitwise OR	rd = rs1 | imm	001
XORI	Immediate Bitwise XOR	rd = rs1 ^ imm	010
All of these operations update the destination integer register, rd			
Compare and Store Immediate (Opcode 3):			
SEQI	Immediate Set Equal	rd = rs1 == imm	000
SNEQI	Immediate Set Not Equal	rd = rs1 != imm	001
SLTI	Immediate Set Less Than	rd = rs1 < imm	010
SGTI	Immediate Set Greater Than	rd = rs1 > imm	011
SLEI	Immediate Set Less Or Equal	rd = rs1 <= imm	100
SGEI	Immediate Set Greater Or Equal	rd = rs1 >= imm	101
All of these operations update the destination integer register, rd. rd is set to 1 if the comparison evaluates to true, and 0 otherwise.			
Jump and Link Register (Opcode 4):			
JALR	Jump and Link Register	rd = pc+1, then pc = rs1+imm	000
This stores the program counter's next location in the destination register, which is x3 by convention, and then sets the program counter to the value in the specified integer register rs1 plus the provided immediate offset.			

Load and Store operations (Type 010):

This type requires two registers. The remaining bits are unused.

Type	Opcode	funct3	Unused	rs1	rs2
3-bit	4-bit	3-bit	12-bit	5-bit	5-bit

The following instructions are Load and Store type operations:

Code	**Common Name**	**Result**	**Funct3**
Load/Store Integer (Opcode 0):			
LW	Load Word	rs2 = *rs1	000
SW	Store Word	*rs1 = rs2	001
Load/Store Floating Point (Opcode 1):			
LWF	Load Word Float	rs2 = *rs1	000
SWF	Store Word Float	*rs1 = rs2	001
The * notation denotes a C dereference. The load operations load the value located at the memory address in register rs1 into rs2. The store operations take the value in rs2 and store it at the memory address in register rs1. The regular instructions use the regular integer registers, while the floating point instructions use floating point registers.			

Jump and Link operations (Type 011):

This type requires an immediate value and one read-register, and has no need for opcode or funct3.

Type	Immediate	rd
3-bit	24-bit	5-bit

The following instructions are Jump type operations:

Code	Common Name	Result
J	Jump and Link	rd = pc+1, then pc += imm
This stores the program counter's next location in the destination register, which is x3 by convention, and then increments the program counter by the immediate value.		

Branching operations (Type 100):

This type requires an immediate value and two read-registers, and has no need for opcode.

Type	funct3	Immediate	rs1	rs2
3-bit	3-bit	16-bit	5-bit	5-bit

The following instructions are Branching type operations:

Code	Common Name	Result	Funct3
BEQ	Branch Equal	if rs1 == rs2: pc += imm	000
BNEQ	Branch Not Equal	if rs1 != rs2: pc += imm	001
BLT	Branch Less Than	if rs1 < rs2: pc += imm	010
BGT	Branch Greater Than	if rs1 > rs2: pc += imm	011
BLE	Branch Less Or Equal	if rs1 <= rs2: pc += imm	100
BGE	Branch Greater Or Equal	if rs1 >= rs2: pc += imm	101
These instructions update the program counter based on the provided immediate value, if the condition is met. Otherwise, the program counter increments as usual. They are used for short, conditional jumps.			

Push/Pop operations (Type 101):

This type requires an immediate value and one register, and has no need for opcode.

Type	funct3	Unused	rs1
3-bit	3-bit	21-bit	5-bit

The following instructions are Push/Pop type operations:

Code	**Common Name**	**Result**	**Funct3**
PUSH	Push	x2 -= 1 *x2 = rs1	000
POP	Pop	rs1 = *x2 x2 += 1	001
These instructions update the sp register, x2, which is the special-purpose stack pointer. Push decrements the sp address by one word (the stack grows downwards), and then sets the value in the memory stored in that register to the value in rs1. Pop sets the value in rs1 to the value stored at the address in sp, and then increments sp by one word.			

Miscellaneous (Type 111):

Type	Opcode	Unused
3-bit	4-bit	

Code	**Common Name**	**Result**	**Opcode**
HLT	Halt	Halts the program	0
RET	Return	pc = ra (return address special register). This functions as a JALR with no immediate, rs1=x3, and rd=x0	1

Any invalid operation will automatically be read as NOP

Registers:

To account for the two differing data types, GIA will feature both a mixed set of 32 registers with support for integer and special-purpose registers as well as a set of 16 floating point registers, and two condition code registers.

The first special-purpose register, x0, will always be hard-coded to refer to the value zero, no matter if the user attempts to alter this value themselves. Next, x1 refers to the current program counter and will only be incremented after every instruction fetch to guarantee that subroutine jumps and returns will be supported. Third, register x2 will always point to the next available space on the top of the stack which would only be able to be altered via the PUSH and POP instructions as to avoid altering the order of the other values on the stack. The final special register of note is x3, the link register, which is a dedicated return address for the next instruction's address. Writes to this register are only valid from J and JALR instructions, any other attempted write will be dropped. This is done as a way to prevent both the simulator and the user from being able to accidentally alter its value via other types of instructions or perceiving it as another general-purpose register for reads and writes. Besides those four registers, the remaining registers in the mixed set belong to the general-purpose registers to account for data handled inside and out of function calls.

The floating-point register set follows a very similar format to the mixed, integer/special-purpose set except for the fact that it only contains sixteen registers, one specifically hard-coded for the zero value and the other fifteen being for general-purpose.

Finally, the last two registers in this simulator are separated from the other registers as they're meant to represent the individual condition codes for both integers and floating points. Due to the different possible conditions to check for with either data type, one register was designed for each data type to avoid any possible points of contention between the two ALUs. To ensure that no-ops and stalls are handled properly and conditions such as over/underflow are avoided, each special condition is given its own unique value to guarantee that the simulator is able to handle each condition in the proper way. In the case of the integer condition codes, the possible conditions are:

Integer Condition Codes:

- Divide by Zero (1)
- Overflow (2)

And for the floating point register, the possible condition codes are:

Floating Point Condition Codes:

- Divide by zero (1)
- Overflow (2)
- Underflow (3)
- NaN (4)
- Infinity (5)

Integer & Special-Purpose Registers

Number	Name	Type	Designation
x0	zero	Special	Zero Register
x1	pc	Special	Program Counter
x2	sp	Special	Stack Pointer
x3	ra	Special	Return Address
x4-x31		General	Registers

Floating Point Registers

Number	Name	Type	Designation
f0	zero	Special	Zero Register
f1-f15		General	FP Registers

Special Registers

Number	Name	Type	Designation
c0		Special	Int Condition Code
c1		Special	FP Condition Code

Special Features:

As previously mentioned, we support floating point values within our simulator. To do this, we will support floating point versions of several of the integer arithmetic instructions, signifying their difference by a unique opcode. We also include separate floating point registers and condition codes to support these instructions.

We will also implement an N-way associative cache, with the level of associativity chosen by the user at runtime. The cache will feature an LRU replacement policy described above.
In terms of cache size, our group will initially test several different cache sizes before ultimately deciding on the best cache size for the scope of our project.

We will also create an in-depth GUI, which displays what is in registers, memory, and cache, while allowing the user to choose to step forward by a specified number of clock cycles or instructions at a time. We will also give further data, such as the cache miss rate and instruction pipeline.

Management Plan:

We are creating this project using C++, and will be using the QT GUI framework. We will be managing both issues and version history via Git (and Github), with Google Docs used for general information sharing and project references.
Our team will communicate via text and email, with regular on-campus meetings to discuss existing work and plan ahead. The structure will be effectively sprint-based, with tasks decided on each week, and verified before moving on to the next steps.

In terms of testing our simulator, our group does not have much experience working with formal C++ testing frameworks, so our testing will be done manually. For instance, our group might find it advantageous to program a trace log to be output for every instruction executed that details the PC, disassembled instruction, source and destination registers. That way, we would be able to debug any potential issues within our simulator by simply walking back through our traces until we eventually came across what specific instruction caused discrepancies in our registers. Additionally, in terms of system-wide testing and debugging, our group could also program several commands designed to state dump and print all data stored within the main memory, cache, and register file between every instruction, allowing us to install set inspection points in our benchmarks.

Project Tasks:

Step 1: Foundational work:
- Create file structure
- Create instruction parsing functionality

Step 2: Basic memory:
- Initialize memory and cache
- Implement 2-way associativity in cache
- Create load/store operations with registers

Step 3: Basic instruction implementation:
- Create functionality for basic instructions

Step 4: Pipeline and GUI:
- Create 5-stage pipeline
- Implement basic GUI

Phase One Complete

Step 5: Full instruction implementation:
- Create functionality for all specified instructions
- Implement interrupt handling to prevent hazards

Step 6: Custom Cache Associativity:
- Modify caching system to be n-way associative, depending on user input

Step 7: Assembler and full GUI:
- Create working assembler
- Implement full GUI

Phase Two Complete